

# AI Datacenter Room Planner — Software Specification

---

Rev 0.1 · 2026-07-25

Generated from Markdown — edit the `.md` source and regenerate (`python3 docs/md2pdf.py`)

# Contents

|                                                             |    |
|-------------------------------------------------------------|----|
| 1 Purpose and Scope .....                                   | 4  |
| 1.1 Purpose .....                                           | 4  |
| 1.2 Relationship to sibling components .....                | 4  |
| 1.3 In scope .....                                          | 4  |
| 1.4 Out of scope .....                                      | 4  |
| 2 Definitions .....                                         | 6  |
| 3 Stakeholders and Primary Use Cases .....                  | 7  |
| 4 Requirements .....                                        | 8  |
| 4.1 Functional requirements .....                           | 8  |
| 4.2 Non-functional requirements .....                       | 11 |
| 4.3 Constraints .....                                       | 12 |
| 4.4 Assumptions .....                                       | 12 |
| 5 Architecture .....                                        | 13 |
| 5.1 Module decomposition .....                              | 13 |
| 5.2 Pipeline .....                                          | 14 |
| 5.3 Execution environments .....                            | 15 |
| 6 Data Design .....                                         | 16 |
| 6.1 Design input (design) .....                             | 16 |
| 6.2 Output document (normative) .....                       | 16 |
| 6.3 Label grammar (normative) .....                         | 18 |
| 7 Algorithm Specification .....                             | 20 |
| 7.1 Traffic matrix — <code>graph.js</code> .....            | 20 |
| 7.2 Partitioning — <code>partition.js</code> .....          | 20 |
| 7.3 Placement — <code>placement.js</code> .....             | 20 |
| 7.4 Pathway routing — <code>pathways.js</code> .....        | 21 |
| 7.5 Port budgeting — <code>fabric.js</code> .....           | 21 |
| 7.6 Media selection .....                                   | 22 |
| 8 Interface Specification .....                             | 23 |
| 8.1 Command line — <code>tools/plan.js</code> .....         | 23 |
| 8.2 Validator — <code>tools/validate_design.py</code> ..... | 23 |
| 8.3 User interface .....                                    | 23 |
| 9 Validation Rule Catalog (normative) .....                 | 25 |
| 10 Verification and Test Strategy .....                     | 27 |
| 10.1 Levels .....                                           | 27 |
| 10.2 Mode matrix (each must build and agree) .....          | 27 |
| 10.3 Requirement traceability (selected) .....              | 27 |
| 10.4 Performance budget (measured, Node 24, Windows) .....  | 28 |
| 11 Risks and Limitations .....                              | 29 |
| 12 Change Control .....                                     | 30 |
| 13 Revision History .....                                   | 31 |

**Status:** Draft **Component:** planner/ **Applies**

**to:** planner v0.1 **Parent specification:**

[SPEC.md](#) (the 64× B200 facility this tool

generalises) **Audience:** datacenter architects, network

engineers, facility engineers, and the developers maintaining this

component



# 1 Purpose and Scope

## 1.1 Purpose

The planner is a **design-time** tool. It converts a set of facility decisions — room size, cooling medium, rack population, switching architecture, power topology — into a **machine-readable rack and cable labeling source of truth** that can be used to order material, print labels, and drive a build.

## 1.2 Relationship to sibling components

| Component         | Question it answers                                                 |
|-------------------|---------------------------------------------------------------------|
| SPEC.md           | What is the 64× B200 cluster? (a fixed, hand-authored design)       |
| planner/          | <b>What should we build, and what does it cost in cable and kW?</b> |
| demo/             | How does a built cluster come up?                                   |
| bootstrap/netbox/ | What is the authoritative record once it is built?                  |

The planner is upstream of NetBox: it produces the design intent that a seed import consumes. It does **not** talk to hardware and has no runtime role.

## 1.3 In scope

Room geometry, aisle layout and rack positioning; cooling mode selection and plant sizing; power chain from utility entrance to PSU cord; fabric topology and port budgeting; pathway routing and cable length derivation; media selection and costing; label generation; design validation; YAML export.

## 1.4 Out of scope

Explicitly **not** provided, and not to be inferred from the output:

| Excluded                        | Rationale                                                                  |
|---------------------------------|----------------------------------------------------------------------------|
| CFD / airflow simulation        | Cooling is modeled as a kW and $\Delta T$ budget only                      |
| Hydraulic pressure, pump curves | Flow is derived from $Q = \dot{m} \cdot c_p \cdot \Delta T$ , nothing more |

| Excluded                                                  | Rationale                                            |
|-----------------------------------------------------------|------------------------------------------------------|
| Short-circuit, coordination, arc-flash study              | Power sizing is capacity and derate only             |
| Code compliance (NEC/IEC/NFPA), seismic, fire suppression | A licensed engineer signs the drawing                |
| Structural floor analysis                                 | Floor loading is a flat kg/m <sup>2</sup> comparison |
| Procurement pricing                                       | Catalog costs are planning figures, not quotes       |
| Runtime/telemetry integration                             | The tool is design-time only                         |

## 2 Definitions

| Term          | Meaning                                                                      |
|---------------|------------------------------------------------------------------------------|
| Position      | A discrete legal rack location on the floor<br>grid (row + slot)             |
| Layout        | A template describing what fills one rack<br>(server SKU, count, companions) |
| Rail          | In a rail-optimized fabric, NIC index $k$ on<br>every GPU node               |
| Routed length | Rack rise + tray run + drop + slack —<br>not straight-line distance          |
| Pathway tier  | An independent tray network: <b>data</b> ,<br><b>power</b> , or <b>fluid</b> |
| Trunk         | A shared pathway run carrying many cables,<br>derived by Steiner tree        |
| Feed          | One side (A or B) of the dual power distribution                             |
| Firm capacity | Capacity remaining after the loss of one<br>redundant unit                   |
| Cut           | Total demand (GB/s) crossing a rack boundary                                 |
| SoT           | Source of truth — the exported YAML document                                 |

### 3 Stakeholders and Primary Use Cases

| ID   | Actor                | Use case                                                        |
|------|----------------------|-----------------------------------------------------------------|
| UC-1 | Datacenter architect | Size a hall for a target GPU count<br>and see whether it fits   |
| UC-2 | Facility engineer    | Compare air vs water cooling for the<br>same fleet              |
| UC-3 | Electrical engineer  | Size entrances, UPS, and<br>distribution; confirm A/B survival  |
| UC-4 | Network engineer     | Compare rail-optimized vs ToR vs EoR on<br>cable cost and reach |
| UC-5 | Install contractor   | Receive a labeled, length-bearing<br>cable schedule             |
| UC-6 | Platform engineer    | Feed design intent into NetBox / CI as<br>a versioned artifact  |

## 4 Requirements

Verification method: **T** = automated test/tool, **D** = demonstration in UI, **I** = inspection of output, **A** = analysis.

### 4.1 Functional requirements

| ID   | Requirement                                                                                                                        | Priority | Verify | Implementation   |
|------|------------------------------------------------------------------------------------------------------------------------------------|----------|--------|------------------|
| FR-1 | The room's width and depth SHALL be user-variable, and the legal rack grid SHALL be re-derived from them                           | Must     | D,T    | js/floor.js      |
| FR-2 | Cold and hot aisle widths, perimeter keep-clear, and clear height SHALL be user-variable                                           | Must     | D      | js/floor.js      |
| FR-3 | The system SHALL report the number of available rack positions and reject a design with more racks than positions                  | Must     | T      | room.capacity    |
| FR-4 | The user SHALL select cooling as <b>air</b> or <b>water</b> ; water SHALL offer direct-to-chip (DLC) and rear-door (RDHx) variants | Must     | D      | js/cooling.js    |
| FR-5 | The cooling selection SHALL impose a per-rack kW ceiling and SHALL flag every rack exceeding it                                    | Must     | T      | cooling.rack_cap |
| FR-6 | The cooling selection SHALL gate which rack layouts are legal; illegal combinations SHALL be reported, never silently substituted  | Must     | T      | cooling.layout   |
| FR-7 | Cooling plant (CRAH / CDU / RDHx / manifold) SHALL be sized, placed with coordinates, and included in the                          | Must     | I      | js/cooling.js    |



| ID    | Requirement                                                                                                                 | Priority | Verify | Implementation             |
|-------|-----------------------------------------------------------------------------------------------------------------------------|----------|--------|----------------------------|
|       | export                                                                                                                      |          |        |                            |
| FR-8  | In-row CDUs SHALL consume real floor positions; failure to place one SHALL be reported                                      | Must     | T      | js/cooling.js              |
| FR-9  | The user SHALL add and remove racks individually, and set the server count per rack                                         | Must     | D      | js/design.js,<br>js/app.js |
| FR-10 | Rack layout SHALL be selectable per rack from a catalog of server SKUs                                                      | Must     | D      | RACK_LAYOUTS               |
| FR-11 | Rack elevations SHALL be generated U-by-U with no overlap and within frame height                                           | Must     | T      | rack.overlap, rack.ru      |
| FR-12 | Switching architecture SHALL be selectable:<br>rail-optimized, top-of-rack, end-of-row                                      | Must     | D,T    | js/fabric.js               |
| FR-13 | Oversubscription and tier count (2 or 3) SHALL be selectable, and the achieved ratio reported when it cannot be met exactly | Must     | T      | fabric.oversubscription    |
| FR-14 | Switch port budgets SHALL be enforced; a design exceeding them SHALL error rather than truncate the cable list              | Must     | T      | fabric.ports               |
| FR-15 | In a rail-optimized fabric, NIC $k$ SHALL land on a rail- $k$ leaf for every host                                           | Must     | T      | fabric.rail<br>(Python)    |
| FR-16 | Utility entrances SHALL be user-variable in count and capacity, and placed on the floor plan                                | Must     | D,I    | js/power.js                |
| FR-17 | UPS model and redundancy (N, N+1, 2N) SHALL be selectable; modules SHALL be sized and placed                                | Must     | D,I    | js/power.js                |

| ID    | Requirement                                                                                                                       | Priority | Verify | Implementation        |
|-------|-----------------------------------------------------------------------------------------------------------------------------------|----------|--------|-----------------------|
| FR-18 | Distribution SHALL be selectable between RPP panels and overhead busway, sized, and placed                                        | Must     | D,I    | js/power.js           |
| FR-19 | Each rack SHALL receive A and B rack PDUs sized so that <b>either side alone</b> carries the full rack load at the derated rating | Must     | T      | power.rack_pdu        |
| FR-20 | Every power connection from entrance to rack PDU SHALL be emitted as a labeled cable; per-PSU cords SHALL be optional             | Must     | I      | js/power.js           |
| FR-21 | A and B power runs SHALL be routed to minimise shared pathway segments; residual sharing SHALL be reported                        | Should   | T      | power.diversity       |
| FR-22 | Cable lengths SHALL be <b>routed</b> lengths derived from room geometry, not straight-line distances                              | Must     | A,I    | js/pathways.js        |
| FR-23 | Cable medium SHALL be selected as the cheapest option whose reach covers the routed length; unreachable links SHALL error         | Must     | T      | cable.reach           |
| FR-24 | Every cable SHALL carry a unique label; duplicates SHALL error                                                                    | Must     | T      | cable.duplicate_label |
| FR-25 | No port SHALL be cabled more than once                                                                                            | Must     | T      | fabric.port_conflict  |
| FR-26 | The export SHALL be a single YAML document containing room, cooling, power, fabric, racks with elevations, devices,               | Must     | I      | js/yaml.js            |

| ID    | Requirement                                                                                        | Priority | Verify | Implementation                |
|-------|----------------------------------------------------------------------------------------------------|----------|--------|-------------------------------|
|       | cables, media, bundles, label conventions, optimization report, totals, and validation             |          |        |                               |
| FR-27 | Rack placement SHALL be solver-assigned, and the user SHALL be able to pin any rack by dragging it | Must     | D      | js/placement.js, js/render.js |
| FR-28 | The tool SHALL report what each solver achieved against a stated baseline                          | Must     | I      | optimization                  |
| FR-29 | The same pipeline SHALL be runnable headlessly and produce identical output to the UI              | Must     | T      | tools/plan.js                 |
| FR-30 | An independent validator SHALL re-derive the checks from the exported YAML alone                   | Must     | T      | tools/validate_design.py      |

## 4.2 Non-functional requirements

| ID    | Requirement                                                                                                                 | Target                    | Verify                   |
|-------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------|--------------------------|
| NFR-1 | <b>Determinism</b> — identical design produces byte-identical YAML                                                          | Always                    | T (cmp of two runs)      |
| NFR-2 | <b>Interactivity</b> — a re-solve at reference scale completes fast enough to feel live                                     | $\leq 250$ ms at 12 racks | T<br>(measured 60 ms)    |
| NFR-3 | <b>Scale</b> — usable at hall scale                                                                                         | $\leq 3$ s at 48 racks    | T (measured 1.2 s)       |
| NFR-4 | <b>Zero build step</b> — no bundler, transpiler, or package install to run the UI                                           | Absolute                  | D                        |
| NFR-5 | <b>Zero runtime dependencies</b> — browser only; headless needs Node $\geq 18$ , validator needs Python $\geq 3.9$ + PyYAML | Absolute                  | I                        |
| NFR-6 | <b>Offline</b> — the UI SHALL function from file:// with no network access                                                  | Absolute                  | D                        |
| NFR-7 | <b>Portability</b> — Linux, macOS, Windows                                                                                  | All three                 | T (developed on Windows) |
| NFR-8 | <b>Auditability</b> — every emitted number traceable                                                                        | Absolute                  | I                        |

| ID     | Requirement                                                                                        | Target   | Verify |
|--------|----------------------------------------------------------------------------------------------------|----------|--------|
|        | to a catalog figure or a stated formula                                                            |          |        |
| NFR-9  | <b>Honest failure</b> — an infeasible design SHALL be built and reported, never silently corrected | Absolute | T      |
| NFR-10 | <b>Independence of verification</b> — the two validators SHALL NOT share code                      | Absolute | I      |

### 4.3 Constraints

| ID  | Constraint                                                                                                                 |
|-----|----------------------------------------------------------------------------------------------------------------------------|
| C-1 | Implementation is ES2020 JavaScript with no modules (plain <code>&lt;script&gt;</code> tags) so <code>file://</code> works |
| C-2 | Modules attach to a single DCP global namespace; no bundler-specific syntax                                                |
| C-3 | All randomness derives from <code>optimizer.seed</code> ; no solver may call <code>Math.random()</code>                    |
| C-4 | Catalog values are planning figures; changing them is a catalog edit, not a code change                                    |
| C-5 | The YAML emitter is hand-rolled — no dependency is permitted in the browser path                                           |
| C-6 | Row pitch is uniform per design (the widest frame in use), matching how a real row is framed                               |

### 4.4 Assumptions

| ID  | Assumption                                                                       |
|-----|----------------------------------------------------------------------------------|
| A-1 | Rows are laid out back-to-back around a hot aisle, front-to-front on cold aisles |
| A-2 | Overhead pathway; data, power, and coolant occupy separate tiers                 |
| A-3 | Cross-aisle tray ladders exist at row ends and every 6 slot columns              |
| A-4 | IT load is on UPS; mechanical plant is on the utility/generator, not the UPS     |
| A-5 | Dual-corded IT: A and B share load normally, each sized for 100%                 |
| A-6 | Power factor $\approx 1.0$ for modern IT PSUs                                    |

## 5 Architecture

### 5.1 Module decomposition

| Module                    | Responsibility                                     | Depends on            |
|---------------------------|----------------------------------------------------|-----------------------|
| <code>util.js</code>      | Seeded PRNG, min-heap, rounding, grouping          | —                     |
| <code>catalog.js</code>   | Hardware, media, cost, layout, architecture tables | —                     |
| <code>design.js</code>    | Design state, defaults, rack add/remove, legality  | catalog, util         |
| <code>floor.js</code>     | Room geometry → row grid → legal positions         | catalog, util         |
| <code>graph.js</code>     | Logical topology and traffic matrix                | util                  |
| <code>partition.js</code> | Multilevel KL/FM partitioning                      | util                  |
| <code>placement.js</code> | QAP placement (SA / GA)                            | util                  |
| <code>pathways.js</code>  | Tray graph, A*, Yen, Steiner, tray fill            | util                  |
| <code>fabric.js</code>    | Switch sizing, port budgets, logical links         | catalog, util         |
| <code>cooling.js</code>   | Cooling plant sizing, siting, coolant runs         | catalog, util         |
| <code>power.js</code>     | Power chain sizing, siting, power runs             | catalog, util         |
| <code>validate.js</code>  | In-process constraint checking                     | catalog, util, design |
| <code>build.js</code>     | <b>Pipeline orchestration</b>                      | all of the above      |
| <code>yaml.js</code>      | Document assembly and YAML emission                | catalog, util         |
| <code>render.js</code>    | SVG floor plan, elevations, tables, reports        | catalog, util, floor  |
| <code>app.js</code>       | Control binding, state, events                     | all                   |

**Dependency rule:** solver modules (`graph`, `partition`, `placement`, `pathways`) SHALL NOT depend on `catalog`, `design`, or any rendering module. They operate on plain numeric structures and are independently testable.

`build.js` is a pure function of (`design`) → `model`. No stage mutates an earlier stage's output.

## 5.2 Pipeline

design

```

|
|→ floor.plan ..... room → rows → legal positions
|→ graph.buildFleet ..... racks → logical servers
|→ graph.trafficMatrix ..... TP/PP/DP → demand edges
|→ partition.partition ..... servers → racks          (minimise cut)
|→ fabric.plan ..... switches, ports, links
|→ elevate ..... U assignment, kW, weight
|→ placement.place ..... racks → positions          (minimise  $\sum F \cdot D$ )
|→ cooling.plan ..... plant sizing + siting
|→ power.plan ..... entrance-UPS-dist-PDU
|→ pathways ..... Steiner trunks, then A* for the rest
|→ media selection + costing
|→ validate.check
|→ yaml.toDocument

```

### Ordering constraints (normative):

- Partitioning **MUST** precede fabric construction (server homing sets leaf demand).
- Fabric construction **MUST** precede placement (the cable-count matrix is an input to the placement objective).
- Elevation **MUST** precede OOB cabling (port demand depends on the device list).
- Placement **MUST** precede cooling and power siting (both need coordinates).
- Trunk (Steiner) routing **MUST** precede point-to-point routing, so trunks receive uncongested pathway.
- A-feed routing **MUST** precede B-feed routing (diversity is computed against A).

### 5.3 Execution environments

| Environment       | Entry                    | Notes                                       |
|-------------------|--------------------------|---------------------------------------------|
| Browser           | index.html               | Plain scripts, file:// capable              |
| Node $\geq$ 18    | tools/plan.js            | Requires the same js/modules; no shims      |
| Python $\geq$ 3.9 | tools/validate_design.py | Reads YAML only; shares no code with the JS |

## 6 Data Design

### 6.1 Design input (design)

Persisted as JSON; accepted by `tools/plan.js` as its input file.

| Group     | Keys                                                                                                                                                                                                                      |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| meta      | name, room, tenant, description                                                                                                                                                                                           |
| room      | width_m, depth_m, clear_height_m, tile_m,<br>perimeter_m, cold_aisle_m, hot_aisle_m,<br>raised_floor,<br>floor_capacity_kg_m2,<br>tray_height_m, power_tray_height_m,<br>data_tray_runs, power_tray_runs, fluid_tray_runs |
| cooling   | mode, water_type, crah_model, cdu_model,<br>rdhx_model, redundancy, supply_c, return_c,<br>containment,<br>racks_per_cdu, air_kw_per_rack_cap                                                                             |
| power     | volts, phases, entrances, entrance_kw,<br>entrance_side, ups_model, ups_redundancy,<br>distribution,<br>rpp_model, busway_model, rack_pdu_model,<br>breaker_derate,<br>emit_device_cords, pue_target                      |
| fabric    | arch, oversubscription, tiers, leaf_model,<br>spine_model, super_model, oob_model, pod_racks,<br>emit_oob                                                                                                                 |
| workload  | tp_size, pp_size, dp_replicas, collective,<br>base_affinity                                                                                                                                                               |
| optimizer | seed, partition, placement, anneal_iters,<br>anneal_start_t,<br>anneal_end_t, objective_traffic_weight,<br>routing, congestion_weight, bend_penalty_m,<br>bundle,<br>slack_m                                              |
| racks[]   | id, name, layout, rack_type, servers,<br>pinned                                                                                                                                                                           |

### 6.2 Output document (normative)

schema\_version: 1. Top-level keys, in emission order:



| Key                | Type | Contents                                                                 |
|--------------------|------|--------------------------------------------------------------------------|
| schema_version     | int  | Document version; consumers MUST reject unknown majors                   |
| generator          | map  | Tool, pipeline summary, seed, determinism flag                           |
| site               | map  | Name, room, tenant, description                                          |
| room               | map  | Dimensions, aisles, row grid, positions, floor rating, pathway tiers     |
| cooling            | map  | Mode, ceiling, capacity, $\Delta T$ , flow, units[] with coordinates     |
| power              | map  | entrances[], ups, distribution, rack_pdus[]                              |
| fabric             | map  | Architecture, port split, oversubscription requested vs achieved, counts |
| racks[]            | list | Frame, position, totals, elevation[] (U, RU, name, model, role, kW)      |
| devices[]          | list | Flat index: name, role, model, rack, U, kW, ports, rail, power source    |
| cables[]           | list | <b>Normative build artifact</b> — see 6.3                                |
| media              | map  | Reach, OD, and cost of each medium used, so the file re-checks itself    |
| bundles[]          | list | Trunk id, cable count, tray segments, trunk length                       |
| labels.conventions | map  | Pattern → meaning                                                        |
| optimization       | map  | Per-solver achieved vs baseline                                          |
| totals             | map  | Rollups by class and by medium, loads, densities                         |
| validation         | map  | ok, counts, and every finding                                            |

Cable record:

```

- label: R0-worker001      # unique across the document
  class: fabric             # fabric | oob | mgmt | power | coolant
  media: aoc-400g          # key into `media`
  length_m: 24.7           # routed, includes slack
  speed_gbps: 400

```

```

feed: A                # power only

rail: 0                # fabric only

bundle: T-spine1       # trunk group, if any

bends: 3

a: { device: worker001, port: rail0, rack: GPU-01 }

b: { device: leaf-rail0, port: Ethernet1, rack: NET-01 }

status: planned

```

### Invariants (normative):

1. label is unique across `cables[]`.
2. (device, port) appears at most once across all cable endpoints.
3. Every endpoint device resolves to `devices[]`, a `rack_pdu`s[] entry, a cooling/power plant unit, or the reserved name `facility-loop`.
4.  $\text{length\_m} \leq \text{media}[\text{media}].\text{max\_m}$  where a reach limit is defined.
5.  $\text{sum}(\text{cables}[].\text{length\_m}) == \text{totals}.\text{cable\_length\_m}$  ( $\pm 0.5$  m).
6.  $\text{len}(\text{cables}) == \text{totals}.\text{cables}$ .
7.  $\text{sum}(\text{racks}[].\text{elevation}) \text{ device count} == \text{len}(\text{devices})$ .
8.  $\text{sum}(\text{racks}[].\text{totals}.\text{kw}) == \text{totals}.\text{it\_load\_kw}$  ( $\pm 0.5$  kW).

## 6.3 Label grammar (normative)

```

fabric-host  := "R" rail "-" server          ; rail-optimized
              | "H" nic "-" server           ; ToR / EoR

leaf-spine   := "L" leaf-tag "S" spine "-U" n

spine-super  := "S" spine "X" super "-U" n

peer         := "PEER-" switch "-" n

oob          := "OoB-" device                 ; BMC

mgmt         := "MGMT-" device                 ; server mgmt0

switch-mgmt  := "MA1-" device                  ; Management1 / ZTP

oob-uplink   := "OoBU-" switch "-" n

power-entry  := "PE-" feed "-" ups

power-feeder := "PF-" feed "-" unit

```

```
power-whip    := "PW-" feed n "-" rack
power-tap     := "PT-" feed n "-" rack      ; busway
power-cord    := "PC-" feed "-" device "-P" n
coolant-sec   := "CW-" ("S"|"R") "-" rack
coolant-pri   := "CF-" ("S"|"R") "-" unit

leaf-tag      := rail                      ; one leaf per rail
               | rail "g" group            ; rail split across leaves
               | rack "t" n                 ; ToR
               | "r" row "n" n             ; EoR
               | "u" n                      ; utility leaf
```

Labels extend the family already used in [docs/cabling.md](#); the single-leaf-per-rail case is byte-compatible with it.

## 7 Algorithm Specification

All solvers are seeded from `optimizer.seed`. All are heuristic; each reports its achieved objective against a stated baseline (NFR-8).

### 7.1 Traffic matrix — `graph.js`

**Input:** workload plan, fleet. **Output:** sparse undirected edges (GB/s).

| Edge class                            | Weight                           | Justification                   |
|---------------------------------------|----------------------------------|---------------------------------|
| TP peers (same tensor-parallel group) | 400                              | All-to-all every layer          |
| PP stage boundary                     | 40                               | Activations, point to point     |
| DP replica ring                       | 120                              | Gradient / KV all-reduce        |
| Background ring                       | <code>base_affinity</code> × 400 | Storage, checkpoint, management |

A job consumes `TP × PP × DP` GPUs; a larger fleet runs `floor(ranks / servers_per_job)` instances. Background is a ring, not a clique, to keep the edge set  $O(n)$ .

### 7.2 Partitioning — `partition.js`

**Objective:** minimise edge cut subject to per-rack slot capacity.

**Method:** multilevel — heavy-edge matching coarsening → capacity-aware greedy graph growing → FM refinement with best-prefix rollback at each uncoarsening level. Because racks are typically filled exactly, refinement also performs KL-style swaps with gain  $D_v(t) + D_u(p) - 2 \cdot c(v, u)$ . **Baseline:** sequential round-robin fill. **Complexity:**  $O(\text{passes} \times V \times B)$  per level,  $B$  = boundary size. **Note:** when a TP group fits inside one server the cut is already minimal and the reported improvement is legitimately 0%.

### 7.3 Placement — `placement.js`

**Objective (QAP):**

$$\text{minimize } \sum_{\{i,k\}} \sum_{\{j,l\}} F_{ik} \cdot D_{jl} \cdot X_{ij} \cdot X_{kl}$$

$F$  is the normalised weighted sum of the **traffic** matrix and the **cable-count** matrix, mixed by `objective_traffic_weight`. The cable term is included because  $\sum F_{\text{cable}} \cdot D$  is total cable

length, making the same solver a cable-length minimiser.

**D** is Manhattan with a  $1.35\times$  penalty on cross-row travel.

**Method:** greedy constructive seed (busiest rack to the most central position, then highest-affinity-to-placed)  $\rightarrow$  simulated annealing over swaps and relocations, geometric cooling from `anneal_start_t` to `anneal_end_t`, temperature normalised against the objective's own scale. Optional order-crossover GA with tournament selection and elitism, always followed by an anneal polish. Pinned racks are frozen and applied last. **Delta evaluation:**  $O(n)$  per candidate move.

**Baseline:** sequential assignment.

## 7.4 Pathway routing — `pathways.js`

**Graph:** per tier — a node per (aisle, slot column), cross-aisle rungs at row ends and every 6 columns, plus a drop per rack/equipment attached to its two nearest aisles. Drop length includes the rise from frame top to tray height.

**Edge cost:**

$$w = \text{length} \cdot (1 + \text{congestion\_weight} \cdot \text{fill}) + \text{bend\_penalty} \quad [\text{if axis changes}]$$

**Search:**  $A^*$  over (node, direction) states so bends are chargeable; heuristic is Manhattan distance (admissible; scaled when trunk-discount edges are in play).

**Capacity:** each tier has a tray cross-section  $(w \times h \times \text{fill\_factor} \times \text{tray\_runs})$ ; each routed cable reserves  $\pi/4 \cdot OD^2$ . Fill is committed immediately, making routing order-dependent as in a real install.

**A/B diversity:** the A feed routes first; Yen's K-shortest ( $K=4$ ) generates candidates for B, scored `shared_segments`  $\times 1000 + \text{length}$ . Residual sharing is reported, not hidden.

**Trunking:** per spine, a Steiner tree over the tray graph via the shortest-path heuristic — grow from the root, repeatedly splice in the nearest unconnected terminal via multi-source Dijkstra, with existing tree segments priced at 5% so reuse dominates.

## 7.5 Port budgeting — `fabric.js`

Leaf split: the largest even downlink count `d` with  $d + \text{ceil}(d/os) \leq P$ . Spine count: the smallest

$s \geq \text{ceil}(\text{total\_uplinks} / \text{spine\_downlink\_ports})$  that divides  $\text{uplinks\_per\_leaf}$  evenly, **capped at  $\text{uplinks\_per\_leaf}$**  — a leaf cannot stripe across more spines than it has uplink ports. When the cap binds, the spine port overflow is reported so the user adds a tier, adds pods, or fits a larger spine.

In a 3-tier fabric, spines reserve 25% of their ports for super-spine uplinks before any leaf is attached.

OOB switches per rack are sized from actual port demand

$(2 \times \text{servers} + \text{switches}, \text{plus uplink terminations in the mgmt rack}),$

so access ports never run into the uplink cages.

## 7.6 Media selection

The cheapest medium whose reach  $\geq$  routed length, per speed ladder. No

reach  $\Rightarrow$  `cable.reach` error. Intra-rack cables use

$|\Delta U| \times 44.45 \text{ mm} + \text{dressing}$  and take no additional slack.

---

## 8 Interface Specification

### 8.1 Command line — `tools/plan.js`

```
node planner/tools/plan.js [design.json] [options]
```

```
-o, --output PATH    write YAML to PATH

--set KEY=VALUE      dotted design path override (repeatable)

--report             human-readable summary

--json              machine-readable summary

-h, --help
```

`--report` and `--json` suppress YAML on stdout so machine output stays clean. Unknown `--set` paths are a hard error.

| Exit code | Meaning                                |
|-----------|----------------------------------------|
| 0         | Built, validation found no errors      |
| 1         | Built, validation found $\geq 1$ error |
| (throw)   | Malformed input or unknown design key  |

### 8.2 Validator — `tools/validate_design.py`

```
python planner/tools/validate_design.py PATH [--quiet]
```

| Exit code | Meaning                          |
|-----------|----------------------------------|
| 0         | No findings                      |
| 1         | $\geq 1$ finding                 |
| 2         | File unreadable or not a mapping |

It additionally emits `validation.disagreement` if the document's own `ok` flag contradicts its independent result.

### 8.3 User interface

Three-column layout: controls (left), views (centre), report (right).

| View       | Contents                                                                                                         |
|------------|------------------------------------------------------------------------------------------------------------------|
| Floor plan | Scale SVG in metre space; overlays for racks, data tray fill, power tray fill, kW density; drag to pin, click to |

| View           | Contents                                                    |
|----------------|-------------------------------------------------------------|
|                | inspect                                                     |
| Elevations     | Per-rack U map with device names and kW, plus A/B PDU chips |
| Cable schedule | Filterable, searchable table of every cable                 |
| YAML           | Live preview, copy, download                                |

The floor plan’s SVG `viewBox` is the room in metres, so no pixel-to-metre conversion exists anywhere and dragging maps directly back to coordinates.



## 9 Validation Rule Catalog (normative)

E = error (not buildable as drawn), W = warning, I =

informational. “Both” = independently implemented in the JS and the Python checker.

| Code                 | Sev | Condition                                                   | Checked by |
|----------------------|-----|-------------------------------------------------------------|------------|
| schema.missing       | E   | Required top-level key absent                               | Python     |
| schema.version       | W   | Unexpected schema_version                                   | Python     |
| room.capacity        | E   | Racks exceed available positions                            | JS         |
| room.rows            | E   | No complete row fits the depth                              | JS         |
| room.height          | E   | Clear height cannot carry the power tray                    | JS         |
| room.cold_aisle      | W   | Cold aisle < 1.2 m                                          | JS         |
| room.hot_aisle       | W   | Hot aisle < 0.9 m                                           | JS         |
| room.floor_load      | E/W | Rack kg/m <sup>2</sup> over (E) or within 15% of (W) rating | JS         |
| room.collision       | E   | Two racks share a row/slot                                  | Python     |
| rack.ru              | E   | Equipment exceeds frame height                              | Both       |
| rack.overlap         | E   | Two devices occupy the same U                               | Both       |
| rack.bounds          | E   | Device spans outside the frame                              | Python     |
| rack.totals          | E   | Stated totals disagree with the elevation                   | Python     |
| rack.duplicate       | E   | Duplicate rack name                                         | Python     |
| cooling.rack_cap     | E   | Rack kW over the mode's per-rack ceiling                    | Both       |
| cooling.layout       | E   | Layout has no variant for the selected mode                 | JS         |
| cooling.capacity     | E   | Plant capacity below IT load                                | Both       |
| cooling.headroom     | W   | < 10% cooling headroom                                      | JS         |
| cooling.loop         | E   | Liquid rack without exactly 2 coolant runs                  | Python     |
| cooling.delta_t      | E   | Non-physical $\Delta T$                                     | Python     |
| cooling.residual_air | I   | DLC still leaves load to the air path                       | JS         |
| power.entrance       | E   | A service cannot carry the hall alone                       | Both       |
| power.ups            | E   | UPS firm capacity below IT load                             | Both       |

|             | Code                                | Sev | Condition                                                  | Checked by |
|-------------|-------------------------------------|-----|------------------------------------------------------------|------------|
| power.      | rpp / power.distributionE           |     | Distribution unit over its derated rating                  | Both       |
|             | power.busway                        | E   | Busway run over capacity                                   | JS         |
|             | power.poles                         | E   | Breaker poles exhausted                                    | Both       |
|             | power.rack_pdu                      | E   | One side cannot carry the rack alone, or a feed is missing | Both       |
|             | power.redundancy                    | W   | Single feed — no A/B diversity                             | Both       |
|             | power.diversity                     | W   | B runs share tray segments with A                          | JS         |
|             | power.totals                        | E   | Rack loads disagree with stated IT load                    | Python     |
|             | fabric.ports                        | E   | Cables exceed a switch's front panel                       | Both       |
| fabric.     | port_conflict / cable.port_conflict | E   | A port is cabled more than once                            | Both       |
|             | fabric.rail                         | E   | NIC $k$ cabled to a non-rail- $k$ leaf                     | Python     |
| fabric.     | oversubscription                    | I   | Achieved ratio differs from requested                      | JS         |
|             | cable.unroutable                    | E   | No pathway between endpoints                               | JS         |
|             | cable.reach                         | E   | Length exceeds the medium's reach                          | Both       |
| cable.      | duplicate_label                     | E   | Label reused                                               | Both       |
|             | cable.endpoint                      | E   | Endpoint resolves to nothing                               | Python     |
|             | cable.media                         | W   | Medium absent from the media table                         | Python     |
|             | cable.length                        | W   | Zero-length cable                                          | Python     |
|             | tray.fill                           | E/W | Tray over 100% (E) or over 80% (W)                         | JS         |
|             | totals.*                            | E   | Rollups disagree with the listed records                   | Python     |
|             | workload.tp_split                   | W   | TP peers split across racks                                | JS         |
| validation. | disagreement                        | E   | Document verdict contradicts the checker                   | Python     |

## 10 Verification and Test Strategy

### 10.1 Levels

| Level       | Scope                                                             | Mechanism                                 |
|-------------|-------------------------------------------------------------------|-------------------------------------------|
| Pipeline    | design → model across the mode matrix                             | tools/plan.js --json, assert error counts |
| Contract    | Exported YAML against §6.2 invariants                             | tools/validate_design.py                  |
| Cross-check | JS verdict vs Python verdict                                      | Disagreement is itself an error           |
| Determinism | Two runs, same design                                             | Byte comparison                           |
| Geometry    | Racks in bounds, finite, no shared positions                      | Node assertion script                     |
| UI          | Render entry points, element/CSS/script wiring                    | DOM-shim<br>smoke script                  |
| Visual      | Floor plan, elevations, schedule, YAML, mode toggles, drag-to-pin | Browser demonstration                     |

### 10.2 Mode matrix (each must build and agree)

`cooling.mode` ∈ {air, water} × `cooling.water_type` ∈ {dlc, rdhx} ×  
`fabric.arch` ∈ {rail-optimized, tor, eor} × `fabric.tiers` ∈ {2,3} ×  
`power.distribution` ∈ {rpp, busway} ×  
`fabric.oversubscription` ∈ {1,2,4} ×  
`optimizer.placement` ∈ {anneal, genetic, greedy, sequential}.

### 10.3 Requirement traceability (selected)

| Requirement | Evidence                                                                                           |
|-------------|----------------------------------------------------------------------------------------------------|
| FR-3        | <code>room.capacity</code> fires when racks > positions                                            |
| FR-5, FR-6  | Air + DLC layout ⇒ 16 errors across <code>cooling.rack_cap</code> / <code>cooling.layout</code>    |
| FR-14       | 21 GPU racks on 64-port spines ⇒ <code>fabric.ports</code> ; resolved by 128-port spine or 3 tiers |
| FR-19       | 101 kW rack ⇒ 3 PDUs per side; <code>power.rack_pdu</code> fires if reduced                        |

| Requirement  | Evidence                                                                |
|--------------|-------------------------------------------------------------------------|
| FR-21        | <code>power.diversity</code> counts residual shared segments            |
| FR-23        | Media distribution shifts DAC→AOC when switching<br>ToR→rail-optimized  |
| FR-24, FR-25 | Duplicate label / port conflict checked in both validators              |
| FR-29        | UI and CLI share <code>js/</code> ; both emit the same document         |
| FR-30        | Python checker found a real defect (coolant runs terminating on a rack) |
| NFR-1        | Two consecutive exports compare identical                               |
| NFR-9        | Undersized network racks produce errors, not silent<br>truncation       |

#### 10.4 Performance budget (measured, Node 24, Windows)

| Racks | GPUs  | Cables | Build    | YAML  | Size   |
|-------|-------|--------|----------|-------|--------|
| 4     | 128   | 422    | 23 ms    | 5 ms  | 138 KB |
| 12    | 576   | 1,628  | 60 ms    | 10 ms | 512 KB |
| 24    | 960   | 2,622  | 282 ms   | 16 ms | 830 KB |
| 48    | 2,240 | 5,840  | 1,205 ms | 29 ms | 1.8 MB |

Growth is dominated by QAP annealing ( $O(\text{iterations} \times \text{racks})$ ) and per-cable A\*. Beyond ~48 racks, reduce `anneal_iters` or partition the hall into pods.

## 11 Risks and Limitations

| ID  | Risk                                           | Impact                          | Mitigation                                                                |
|-----|------------------------------------------------|---------------------------------|---------------------------------------------------------------------------|
| R-1 | Heuristic solvers may miss a better layout     | Suboptimal cable cost           | Baseline comparison is always reported; seed is adjustable                |
| R-2 | Catalog figures drift from vendor reality      | Mis-sized plant                 | Catalog is a single isolated file with cited units                        |
| R-3 | Routing order-dependence                       | Different fill under reordering | Deterministic order; fill is reported and validated                       |
| R-4 | Tray model is a simplification                 | Optimistic congestion           | Fill reported at 80%/100% thresholds; <code>tray_runs</code> configurable |
| R-5 | Output could be mistaken for a stamped drawing | Compliance risk                 | \$1.4 exclusions; README “What this is not”                               |
| R-6 | Large halls exceed interactive latency         | Poor UX                         | Documented budget; pod the hall                                           |

## 12 Change Control

---

- Catalog changes (SKUs, costs, reach) do **not** require a spec revision.
  - Changes to the output document shape require a `schema_version` bump and a revision entry here.
  - New validation codes require a row in §9.
  - New user-facing capability requires an FR row in §4.1 and traceability in §10.3.
-

### 13 Revision History

| Version | Date       | Notes                                                                                                                                                                       |
|---------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.1     | 2026-07-25 | Initial specification: room, air/water cooling, rack add/remove, layout + switching architecture, power entrances/UPS/PDU, optimization pipeline, YAML SoT, dual validation |